



Faculty of Computing

Department of Computing

Laboratory Manual

CS-250 : Data Structure and Algorithms

Class: BSCS 16AB • Semester: Fall 2026

Experiment No. 04

Runtime Complexity Analysis

Date	02 October 2026
Time	09:00 AM–11:50 AM & 2:30 PM–5:00 PM
Instructor(s)	Dr Faisal Shafait
Lab Engineer	Hamza Fayyaz
Mapped CLO	CLO2 : Solve a given real time problem by applying appropriate data structure and algorithm. CLO3 : Practice programs using the latest IDEs ensuring testing, documentation and packaging of programs as per standards practices applicable to the software industry. CLO4 : Demonstrate commitment to professional ethics by following engineering norms applicable to the software industry.:

1 Introduction

Runtime complexity describes how the work performed by an algorithm grows with input size n . This lab analyzes familiar array and linked-list operations using measured execution time and Big-O notation.

2 Objectives

Analyze simple algorithms, identify $O(1)$, $O(n)$ and $O(n^2)$ behavior, and compare runtime costs of arrays, singly linked lists and doubly linked lists.

3 Tools and Software Requirements

Software: Visual Studio C++, VS Code

Helping Material: Lecture slides, Text book, previous linked-list lab code

4 Lab Tasks

Reuse previous linked-list code where required. Use `rand()` to generate test data and C++ `<chrono>` to measure execution time in milliseconds. Record both measured time and theoretical Big-O complexity where requested.

Task 1: Basic Runtime Analysis

For each code fragment, state the time complexity in Big-O notation and briefly justify your answer. Where applicable, execute the code for different input sizes and record the running time in milliseconds.

Listing 1. A

```
1 cout << arr[0];
```

Listing 2. B

```
1 for (int i = 0; i < n; i++)
2     cout << arr[i] << " ";
```

Listing 3. C

```
1 for (int i = 0; i < n; i++)
2     for (int j = 0; j < n; j++)
3         cout << arr[i] + arr[j];
```

Listing 4. D

```
1 Node* temp = head;
2 while (temp != nullptr) {
3     temp = temp->next;
4 }
```

Code	Complexity	Reason
A		
B		
C		
D		

Task 2: Singly Linked List Traversal

Using your singly linked list, create lists of 100, 1,000, 10,000 and 100,000 nodes and fill them with random integers using `rand()`. Search for the first, middle and last node. For each case, record the number of nodes examined and the running time in milliseconds. State the time complexity in Big-O notation.

n	First (ops / ms)	Middle (ops / ms)	Last (ops / ms)
100			
1,000			
10,000			
100,000			

Task 3: Dynamic Array vs Singly Linked List

Create a dynamic array and a singly linked list of the same size and fill both with the same random integers using `rand()`. For access first, access k th, access last and search by value, record the running time in milliseconds and state the time complexity in Big-O notation for both structures.

Operation	Dynamic Array (Big-O / ms)	Singly Linked List (Big-O / ms)
Access first		
Access k th		
Access last		
Search by value		

Task 4: Cost of Finding a Node

Create a singly linked list and fill it with random integers using `rand()`. (a) Delete the last node when only head is available. (b) Delete a node when pointers to that node and its previous node are already available. For both cases, record the running time in milliseconds, state the time complexity in Big-O notation, and briefly explain why they differ.

Task 5: Singly vs Doubly Linked List

Create singly and doubly linked lists containing the same random integers using `rand()`. Assume a pointer to the current node is already available. Perform Next, Previous, Insert After, Insert Before and Delete. For each operation, record the running time in milliseconds, state the time complexity in Big-O notation, and give a short reason.

Operation	Singly Linked List	Doubly Linked List
Next		
Previous		
Insert After		
Insert Before		
Delete		

Final Question: How does the extra prev pointer in a doubly linked list trade additional memory for faster operations? Give two examples.

5 Deliverables

Complete the solution sections and submit a single Word document on LMS. Show your implementation and outputs during lab grading.

6 Lab Rubrics

Assessment	Doesn't meet Expectation (1–2)	Meets Expectation (3–4)	Exceeds Expectation (5)	Marks
Software Problem Realization (CLO2)	Unable to identify or analyze the required runtime complexity.	Identifies most complexities with some guidance.	Correctly analyzes operations and justifies runtime complexity.	
Software Tool Usage (CLO3)	Code is incomplete or does not produce required results.	Code is mostly functional with minor issues.	Code is complete, functional, documented and produces correct results.	
Ethics and Adherence to Laboratory Safety Rules (CLO4)	Does not follow professional/lab norms.	Generally follows professional and laboratory norms.	Consistently follows professional ethics, lab decorum and safety practices.	